

文書意味正規化ゲートウェイによる 視覚的プロンプトインジェクション対策

論文・レポート PDF を LLM へ安全に投入するための
定義、脅威モデル、正規化方式、数式転写サンドボックス設計

本レポートの中心提案

PDF を単にテキスト抽出または画像化して LLM へ渡すのではなく、文書領域ごとに必要な情報量を制限し、背景・文字・表・数式・図・写真を異なる経路で正規化する。特に数式については、元画像を最終 LLM へ渡さず、権限を持たない**数式転写専用 VLM**に一時的に提示し、**数学用 LaTeX** だけへ圧縮してから検証・再レンダリングする。

作成日: 2026 年 8 月 11 日

対象実装: documentation.wl (2026-08-11 添付版)

本文中の「Document Semantic Canonicalization Gateway (DSCG)」および「Formula Transcription Sandbox (FTS)」は、本レポートで提案方式を記述するために定義した名称であり、既存研究における標準名称ではない。

Contents

1 要旨	4
2 目的と適用範囲	4
2.1 目的	4
2.2 非目標	5
3 用語と定義	5
3.1 プロンプトインジェクション	5
3.2 PDF 隠しテキスト型インジェクション	5
3.3 敵対的画像と visual adversarial example	5
3.4 Image hijack	5
3.5 不可視 visual prompt injection	6
3.6 ステガノグラフィック・プロンプトインジェクション	6
3.7 OCR adversarial attack	6
3.8 文書正規化と semantic bottleneck	6
4 脅威モデル	6
4.1 攻撃者能力	6
4.2 攻撃目標	7
5 提案方式: Document Semantic Canonicalization Gateway (DSCG)	7
5.1 定義	7
5.2 全体アーキテクチャ	8
5.3 設計原則	8
6 背景平坦化と二値化	8
6.1 なぜ論文背景は有利か	8
6.2 ページ全体の 1-bit 化ではなく背景選択的二値化	9
6.3 背景異常スコア	9
7 領域分割と領域別 semantic squeezing	9
7.1 基本分類	9
7.2 Feature squeezing との関係	10
8 数式転写サンドボックス (Formula Transcription Sandbox; FTS)	10
8.1 再レンダリング方式の難点	10
8.2 FTS の定義	10

8.3	複数正規化ビューによる一致検査	11
8.4	数学 LaTeX ホワイトリスト	11
8.5	Round-trip verification	11
8.6	インライン数式は自由テキスト再生成を避ける	12
9	表・図・写真の処理	12
9.1	表	12
9.2	線図・フローチャート	12
9.3	グラフ	12
9.4	写真・顕微鏡像・医用画像	12
10	Provenance と Capability Gate	13
10.1	サニタイズだけに安全性を依存しない	13
10.2	Capability separation	13
11	documentation.wl への適用	13
11.1	現行コードで確認できる数式経路	14
11.2	推奨変更: 画像 VLM フォールバックの隔離	14
11.3	インライン数式関数の改善	14
11.4	提案 API	14
12	推奨アルゴリズム	15
12.1	ページ処理	15
12.2	数式処理	15
13	評価方法	16
13.1	評価軸	16
13.2	ベンチマーク構成案	16
13.3	Ablation	17
14	限界と Adaptive Attack	17
14.1	背景を消せば攻撃は foreground へ移る	17
14.2	二値画像に対しても攻撃は理論上可能	17
14.3	transform-aware attack	17
14.4	写真の semantic corruption	18
14.5	認識器の共通モード失敗	18
15	実装優先順位	18
15.1	Phase 1: 低コスト・高効果	18
15.2	Phase 2: 数式経路の強化	18

15.3 Phase 3: 文書全体の semantic routing	18
16 方式の利点とトレードオフ	18
17 結論	19
A 数学 LaTeX サブセットの設計例	19
B 実装チェックリスト	20
C 参考文献	20

1 要旨

LLM や Vision-Language Model (VLM) に外部 PDF を直接与える方式では、PDF の可視内容だけでなく、透明文字、非表示オブジェクト、メタデータ、不可視に近い画素摂動などがモデル入力へ混入し得る。間接プロンプトインジェクションは、外部コンテンツ内の命令を LLM が「データ」ではなく「実行すべき指示」として扱うことで成立する [1]。近年は、VLM の高次元な画像入力を利用し、人間にはほぼ知覚できない摂動でモデル挙動を誘導する image hijack や visual adversarial jailbreak が実証されている [3, 2]。2026 年には不可視に近い visual prompt injection を明示的に扱う研究も報告されている [7]。

本レポートでは、論文・技術レポートの画像表現が一般の自然画像よりはるかに低エントロピーであることを利用する。典型的な学術文書では、背景はほぼ単色、本文は少数色の文字、表は罫線と文字、数式は白地に黒い glyph、線図・グラフも限定色である。したがって、文書理解に不要な画素自由度を決定論的に削除し、最終 LLM へ到達する情報を「意味的に必要な成分」に限定できる。

本レポートはこの入力境界を**文書意味正規化ゲートウェイ (Document Semantic Canonicalization Gateway; DSCG)**と呼ぶ。DSCG は、(1) 信頼できる rasterizer、(2) 背景平坦化と異常検出、(3) 領域別 quantization / binarization、(4) OCR および構造化、(5) 数式転写サンドボックス、(6) provenance 付与、(7) capability gate、からなる。数式は再レンダリングで精度が落ちやすいため、画像を完全排除するのではなく、画像を読む VLM を**転写専用・無権限**のコンポーネントとして隔離し、出力を数式用 LaTeX 文法へ制限する。

結論 「PDF を画像化して OCR する」だけでも透明テキスト型の攻撃面は大きく削減できる。しかし OCR 自体も敵対的画像の標的になり得る。より強い設計は、**文書が本来必要としない画像自由度を領域ごとに削り、画像由来の情報を狭い構文へ変換し、最終 reasoning/agent LLM には正規化済み表現だけを渡すことである**。安全性の最終保証は入力サニタイズだけに依存せず、文書由来情報にツール権限を与えない capability separation で行う。

2 目的と適用範囲

2.1 目的

本方式の目的は、外部から取得した論文・レポート・技術文書を LLM に利用させる際に、次の三つを同時に満たすことである。

1. **可読情報の維持**: 本文、数式、表、図、写真など、研究文書の意味理解に必要な情報をできるだけ保つ。
2. **不可視チャンネルの縮退**: PDF 内部構造や微小画素変化を利用する隠れた命令・摂動が最終 LLM まで届く通信路を減らす。
3. **権限分離**: 仮に認識器が攻撃に成功しても、その出力だけでファイル操作、外部送信、秘密情報読出し等が実行されないようにする。

対象は主として、学術論文、報告書、仕様書、講義資料など「ページ背景と構造が比較的規則的な文書」である。ポスター、広告、芸術作品、雑誌誌面など、背景そのものに意味がある文書では正規化強度を下げる必要がある。

2.2 非目標

本方式は次を単独で完全に防ぐものではない。

- PDF パーサや画像デコーダのメモリ破壊脆弱性。これは sandbox、更新管理、プロセス分離が必要である。
- 人間にも明確に読める悪意ある命令。これは画素正規化では消えず、通常の indirect prompt injection 対策が必要である。
- 写真内容そのものに自然に組み込まれた命令や、意味的に成立する adversarial content。
- 攻撃者が正規化方式を完全に知り、その後段の認識器まで含めて最適化する adaptive attack に対する数学的完全保証。

3 用語と定義

3.1 プロンプトインジェクション

プロンプトインジェクション (Prompt Injection; PI) とは、LLM が処理する入力中に攻撃者が命令を混入し、開発者・ユーザーが意図した指示階層や処理方針からモデル挙動を逸脱させる攻撃を指す。本レポートでは、特に外部文書、検索結果、Web ページ、メール、PDF 等に埋め込まれた命令が後から LLM へ取り込まれるものを**間接プロンプトインジェクション (Indirect Prompt Injection; IPI)** と呼ぶ [1]。

PI の本質は、LLM アプリケーションが**命令とデータを同じ言語表現として受け取る点**にある。従って「悪意ある文字列を検出する」だけでは不十分であり、信頼境界と権限境界を別に設計する必要がある。

3.2 PDF 隠しテキスト型インジェクション

PDF 隠しテキスト型インジェクションとは、人間が通常の PDF ビューアで見る内容と、PDF テキスト抽出器が取得する内容との差を利用する攻撃である。例として、背景色と同色の文字、極小文字、ページ外配置、透明度を利用した文字、レイヤやオブジェクト順序を悪用した文字がある。学術査読を LLM に行わせる状況で、不可視テキストが評価へ影響し得ることが報告されている [9, 10]。2026 年には、hidden prompt injection を含む PDF を大規模に生成した CrackedPDFs ベンチマークも提案されている [11]。

3.3 敵対的画像と visual adversarial example

画像を $I \in [0, 1]^{H \times W \times C}$ とし、人間から見た意味をほぼ保つ小さな摂動 δ を加えた画像 $I' = I + \delta$ によってモデル出力を攻撃者の目的へ変化させるものを、本レポートでは **visual adversarial example** と呼ぶ。VLM では攻撃目標が単なる分類誤りに限られず、生成応答の誘導、jailbreak、コンテキスト漏洩などへ拡張される [2, 3]。

3.4 Image hijack

Bailey らは、推論時に VLM の振る舞いを制御する敵対的画像を **image hijack** として研究した [3]。重要なのは、画像内に人間が読める命令文が存在しなくても、視覚エンコーダと生成モ

デルの内部表現を通じて行動が誘導され得る点である。

3.5 不可視 visual prompt injection

本レポートでは、攻撃者の命令が画像チャンネルに埋め込まれ、通常の閲覧では認識しにくいにもかかわらず MLLM の instruction-following 挙動を誘導する攻撃を**不可視 visual prompt injection**と総称する。2026 年の Ding らは、bounded text overlay による意味誘導と、視覚・テキスト表現への feature alignment を組み合わせた imperceptible visual prompt injection を報告している [7]。

3.6 ステガノグラフィック・プロンプトインジェクション

Steganographic prompt injection は、空間領域、周波数領域、neural steganography 等を用いて画像に命令情報を隠す攻撃概念である。2025 年には VLM を対象とした報告がある [6]。ただしこの研究領域は image hijack や一般的 adversarial example に比べて新しく、方式・モデル・評価条件によるばらつきが大きい。したがって、本レポートでは「成立し得る追加チャンネル」として扱い、一般化された確立事実とは区別する。

3.7 OCR adversarial attack

OCR を $O(I)$ とすると、人間には元とほぼ同じに読める画像 I' を作り、 $O(I')$ だけを異なる文字列へ変える攻撃を **OCR adversarial attack** と呼ぶ。watermark 風の自然なノイズを付加して OCR 結果を誘導する研究も存在する [5]。従って「PDF を rasterize して OCR したから安全」とは言えない。

3.8 文書正規化と semantic bottleneck

本レポートにおける**文書正規化 (document canonicalization)**とは、文書理解に不要な入力自由度を決定論的または強く制約された変換によって削除し、同じ可視・意味内容を持つ多数の画素表現を少数の標準表現へ写像する操作である。この発想は、bit-depth reduction や spatial smoothing で敵対者の探索空間を縮小する feature squeezing[4] と整合するが、本提案は文書領域の意味構造を利用して領域ごとに異なる squeezing を行う点が異なる。

Semantic bottleneck とは、元画像の高次元な画素列を直接後段へ渡す代わりに、OCR 文字列、表構造、数学 AST、限定色図像など、タスクに必要な低次元表現へ変換する境界をいう。

4 脅威モデル

4.1 攻撃者能力

本レポートでは強めの攻撃者を想定する。攻撃者は配布 PDF の内容を任意に作成でき、PDF オブジェクト、フォント、透明テキスト、画像画素、埋込画像、色、背景、微小摂動を制御できる。また防御パイプラインの概略を知り、JPEG 品質、二値化、resizeなどを想定した adaptive optimizationを試みる可能性がある。

一方、次は信頼できるものと仮定する。

- sandbox 内で動作する PDF rasterizer および画像デコーダ
- canonicalization 実装そのもの
- strict parser / validator
- capability gate と秘密情報アクセス制御

VLM や OCR は**信頼しない認識器**として扱う。すなわち、誤認識や prompt injection による乗っ取りが起き得る前提で、その出力に直接権限を与えない。

4.2 攻撃目標

代表的な攻撃目標は次である。

1. 文書要約、査読、採点、分類を攻撃者に有利な方向へ偏らせる。
2. 「元指示を無視せよ」等によってユーザー指示を上書きする。
3. agent にファイル読出し、外部送信、API 実行等を行わせる。
4. 他の文書や秘密コンテキストをモデル出力に漏洩させる。
5. 数式・表・図の認識結果をわずかに変更し、科学的意味を改変する。

最後の項目は通常の prompt injection より微妙である。例えば $\sum$ を $\prod$ 、 $<$ を $>$ 、 1 を l 、 α を a に誤認させるだけでも、論文理解や再現実験に大きな影響を及ぼす。このため数式経路では「命令を防ぐ」だけでなく**転写忠実度の検証**も必要になる。

5 提案方式: Document Semantic Canonicalization Gateway (DSCG)

5.1 定義

定義 (DSCG) Document Semantic Canonicalization Gateway とは、信頼できない文書を最終 LLM へ渡す前に、ページをレンダリングし、文書領域を意味クラスへ分解し、各領域を必要最小限の表現へ正規化し、その出力に provenance を付与して権限制御へ渡す入力境界である。

DSCG は「攻撃を完全に識別する検出器」ではなく、「攻撃者が利用できる入力次元を減らす変換器」として設計する。可視文書に不要な自由度を積極的に捨てることで、未知の攻撃に対しても攻撃面を狭めることを狙う。

5.2 全体アーキテクチャ

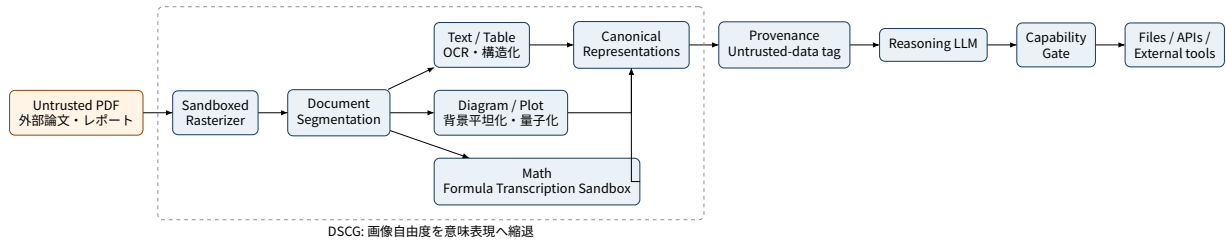


Figure 1: DSCG の概念構成。画像を解釈する認識器と、権限を持つ agent を分離する。

5.3 設計原則

1. **Render first:** PDF 内部の text layer やオブジェクトを直接 LLM へ渡さず、まず信頼できる renderer で可視ページへ投影する。
2. **Flatten low-entropy regions:** 背景や余白など、本来単一色である領域を完全な標準色へ写像する。
3. **Route by semantics:** 本文、数式、表、線図、写真を同じ画像処理で扱わない。
4. **Transcribe, do not obey:** VLM の役割を「画像内容の転写・分類」に限定し、文書内の命令を実行する主体にしない。
5. **Constrain output grammar:** 認識器出力を自由自然言語ではなく、JSON schema、数学 LaTeX、表構造等に制限する。
6. **Reconstruct from trusted primitives:** 可能な領域は信頼できる font/renderer で再構成し、元画素を最終 LLM へ引き継がない。
7. **Separate safety from sanitization:** サニタイズが破られても、capability gate がツール利用や秘密情報アクセスを阻止する。

6 背景平坦化と二値化

6.1 なぜ論文背景は有利か

自然画像と異なり、学術論文の余白や本文背景は多くの場合ほぼ一様な白である。通常の PDF をレンダリングした背景に、ページ全体へ連続的な低振幅 gradient、RGB チャネル間の規則的な揺らぎ、局所的高周波ノイズが存在する必然性は小さい。従って、これらを「保存すべき情報」とみなす必要がない。

ページ画像を $I: \Omega \rightarrow [0, 1]^3$ とする。背景色推定値を b 、画素 x 周囲の局所分散を $V_r(x)$ とし、背景マスクを

$$M_b(x) = \mathbf{1} [\|I(x) - b\|_2 < \tau_c] \cdot \mathbf{1} [V_r(x) < \tau_v]$$

と定義する。背景正規化 C_b は

$$C_b(I)(x) = \begin{cases} (1, 1, 1) & M_b(x) = 1, \\ I(x) & \text{otherwise} \end{cases}$$

とする。これにより背景に埋め込まれた連続値情報は、純白という一つの表現へ圧縮される。

6.2 ページ全体の 1-bit 化ではなく背景選択的二値化

ページ全体を単純閾値で 1-bit 化すると、細い数式記号、アンチエイリアス、薄い罫線、グレースケールのグラフ、写真が壊れる。したがって推奨するのは、**背景領域は完全単色化し、foreground は領域クラスに応じて異なる強度で量子化する**方式である。

領域	推奨正規化	理由
ページ背景・余白	純白または推定標準色へ完全 flatten	高次元な隠れチャンネルをほぼ全廃できる
本文文字	2 値または 2-4 bit grayscale	glyph 輪郭を保持しつつ微小色変化を削る
数式	高解像度 gray + 2 値版を併用	細線を保持し、複数変換間で一致検査できる
表	背景 flatten + OCR/セル構造化	元画素より構造情報の方が重要
線図・グラフ	8-32 色程度の palette 化	色意味を保ちながら連続色自由度を削る
写真・顕微鏡像	resize + JPEG + 必要最小限の色量子化	意味情報が画素にあるため強い二値化は不可

Table 1: 領域別の正規化強度。

6.3 背景異常スコア

正規化は「消す」だけでなく「怪しさを測る」にも使える。背景候補領域 B に対し、例えば

$$A_{bg} = \alpha \text{Var}_{x \in B}(I(x)) + \beta \mathbb{E}_{x \in B} \|\nabla I(x)\|_2 + \gamma E_{hf}(I|_B)$$

を定義する。 E_{hf} は高周波成分のエネルギーである。通常文書の背景統計をコーパスから求め、閾値を超えたページを「攻撃確定」ではなく**追加検証対象**として扱う。

背景に gradient があること自体は攻撃の証明ではない。スキャン紙、照明むら、透け、圧縮、ウォーターマークなど正常理由もある。従って背景異常スコアは拒否判定の単独根拠ではなく、正規化強度の切替・別経路 OCR・人手確認を促すシグナルとして使う。

7 領域分割と領域別 semantic squeezing

7.1 基本分類

DSCG はページを少なくとも次のクラスに分ける。

1. background / margin
2. prose text
3. inline math
4. display math
5. table

6. line diagram / flowchart
7. plot / chart
8. photograph / microscopy / raster scientific image

この分類自体に ML を使ってもよいが、分類器の出力は「どの正規化を適用するか」にのみ使い、外部ツール権限には接続しない。分類が不確実な領域は、弱い正規化と複数経路比較へ回す。

7.2 Feature squeezing との関係

Feature squeezing は、bit depth reduction や smoothing で多数の入力を同一表現へ合流させ、敵対的例の探索空間を縮小する考え方である [4]。DSCG はこれを文書に特化させ、**ページ全体に一つの squeezer を適用するのではなく、意味領域ごとに異なる squeezer を適用する。**

例えば背景なら 24-bit RGB の数百万色を 1 色へ、本文なら連続階調を 2-4 階調へ、グラフなら数千万色を限定 palette へ落とせる。一方、写真では強い squeezing を避ける。この非一様性が、文書品質と攻撃面削減の両立に重要である。

8 数式転写サンドボックス (Formula Transcription Sandbox; FTS)

8.1 再レンダリング方式の難点

数式は、単純 OCR や文字列抽出だけでは再現が難しい。分数、上下添字、積分範囲、行列、piecewise、装飾記号、複雑な括弧構造などは 2 次元配置が意味を持つ。特に Mathematica の Box 構造や PDF 由来の数式では、構造化変換に失敗した場合に、**最終レンダリング画像を VLM へ見せて LaTeX へ転写する方が高精度になる場合がある。**

したがって DSCG では「画像を一切 VLM に渡さない」を要件としない。代わりに、元数式画像を見るコンポーネントを**権限のない転写器**へ隔離する。

8.2 FTS の定義

定義 (FTS) Formula Transcription Sandbox とは、数式領域の正規化済み画像のみを入力とし、出力を許可された数学用 LaTeX 構文または数学 AST に限定し、ファイル・ネットワーク・ツール・秘密コンテキストへアクセスできない VLM 転写環境である。

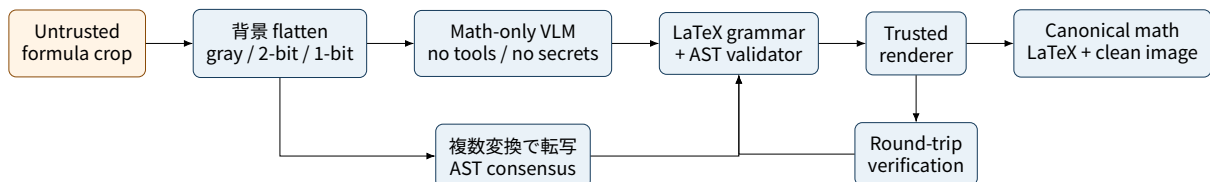


Figure 2: 数式画像を最終 reasoning LLM から隔離する FTS。

8.3 複数正規化ビューによる一致検査

数式 crop I_m から、例えば

$$T_1(I_m) = \text{background-flattened grayscale},$$

$$T_2(I_m) = \text{2-bit grayscale}, \quad T_3(I_m) = \text{adaptive binary}$$

を生成し、それぞれを同一または独立の転写器へ入力する。得られる LaTeX を y_1, y_2, y_3 、parser を P とし、

$$P(y_1) \equiv P(y_2) \equiv P(y_3)$$

であれば高信頼とする。文字列一致ではなく AST 同値を使うことで、空白や表記差を許容できる。

この検査は「敵対的摂動なら必ず不一致になる」という保証ではない。しかし、微小画素差に依存する脆弱な認識を検知する追加シグナルとして有効である。元入力と squeezed 入力のモデル応答差を見る発想は feature squeezing に類似する [4]。

8.4 数学 LaTeX ホワイトリスト

FTS 出力は任意 LaTeX として信用してはならない。TeX は本来プログラミング能力を持つため、実行可能な命令を含み得る。安全な実装では、LaTeX をそのまま shell-enabled TeX へ渡すのではなく、**数学サブセットの parser** へ通す。

許可候補は次のような純粋な数学構文に限定する。

許可候補: `\frac`, `\sqrt`, `\sum`, `\prod`, `\int`, `\partial`, `\lim`, `\left`/`\right`, 限定された matrix 系環境、ギリシャ文字、関係演算子、添字・上付き、括弧、数字、変数、限定的な `\mathrm` / `\operatorname` 等。

原則禁止する例は、ファイル I/O、パッケージ読込、外部画像、特殊命令に関するコマンドである。具体的には `\input`, `\include`, `\write`, `\openout`, `\special`, `\usepackage`, `\includegraphics` 等を拒否する。

より安全には、LaTeX 文字列を一度独自 AST へ parse し、AST から信頼できる renderer 向け LaTeX を再生成する。

8.5 Round-trip verification

転写結果 y を trusted renderer R で画像 $\hat{I}_m = R(y)$ に戻し、正規化済み入力 I'_m と比較する。フォント差のため pixel 一致は使わず、数式構造に関する特徴を比較する。

- connected component 数と大まかな位置
- baseline、上付き・下付きの垂直関係
- fraction bar、radical、括弧の幾何
- 行列の行列数と括弧
- operator 位置と添字範囲
- 別の math recognizer による再転写結果

最終的には $P(y)$ と再転写 AST $P(\hat{y})$ が同値かを確認する。重要論文では一致しない式のみ人手確認へ回せる。

8.6 インライン数式は自由テキスト再生成を避ける

インライン数式を含む段落で、VLM に「段落全文を再現し、数式だけ LaTeX 化せよ」と依頼すると、VLM 出力が自由自然言語となり、攻撃チャンネルが広い。より安全な方式は、本文文字列を信頼側で固定し、VLM には**数式部分だけ**を返させることである。

推奨出力例:

```
{
  "formulae": [
    {"index": 1, "latex": "\\frac{a}{b}"},
    {"index": 2, "latex": "x_i^{(t+1)}"}
  ]
}
```

ホスト側は元の plain text や cell 構造を保持したまま、既知の math slot へ validated LaTeX だけを挿入する。これにより転写 VLM が本文中へ新しい命令文を生成する余地を狭められる。

9 表・図・写真の処理

9.1 表

表は画像として保持する必要がある場合が多い。推奨経路は、背景 flatten、罫線検出、OCR、セル構造推定を行い、最終的に HTML table、Markdown table、CSV 相当の構造へ変換することである。セル結合や数式がある場合は、表セル内の math region のみ FTS へ送る。

9.2 線図・フローチャート

線図は背景を純白化し、色を限定 palette へ量子化する。可能であればテキストラベルを OCR し、矢印・box・edge を構造化する。完全な vectorization が難しい場合でも、背景 flatten + resize + palette quantization だけで不要な画素自由度を大きく削減できる。

9.3 グラフ

グラフでは色が系列識別に必要なため、単純二値化は不適切である。ただし 24-bit 連続色を維持する必要は通常なく、legend と系列の識別を維持する範囲で palette 化できる。小さなマーカーや error bar を失わないよう、downsampling と palette 化の順序を評価する必要がある。

9.4 写真・顕微鏡像・医用画像

写真系が最も難しい。画素の微妙な濃淡そのものが科学的意味を持つ場合、強い squeezing は情報損失になる。ここでは次の多層防御を用いる。

- メタデータ除去、標準色空間への変換

- 適度な resize
- JPEG 再圧縮または標準化 PNG 再エンコード
- 必要に応じて bit-depth / palette の軽い削減
- 複数変換で VLM 記述の consistency を見る
- 高リスク用途では adversarial purification を追加する

VLM 向け adversarial purification として diffusion model を用いる DiffCAP のような研究も提案されている [8]。ただし再生成型 purification は科学画像の定量情報を改変し得るため、顕微鏡像・医用画像等では「安全化のために内容を変えてしまう」リスクを別途評価すべきである。

10 Provenance と Capability Gate

10.1 サニタイズだけに安全性を依存しない

背景 flatten や FTS は攻撃面を減らすが、未知の攻撃を 100% 排除できる保証はない。従って、最終 LLM へ渡すデータには明示的な provenance を持たせる。

概念的には次のような型である。

```
{
  "source": "external_document",
  "trust": "untrusted_content",
  "kind": "formula_transcription",
  "content": "...validated representation..."
}
```

重要なのは、この tag を自然言語プロンプトの注意書きだけで実現しないことである。アプリケーション側で「external document 由来の値は policy 変更や tool permission 変更の根拠にできない」という制約を持つ。

10.2 Capability separation

文書理解モデル M_d が侵害されたとしても、直接ツール T を呼べないようにする。ツール実行は policy G を通し、

$$G(a, \text{provenance}, \text{user intent}, \text{capability}) \in \{\text{Permit}, \text{Deny}, \text{NeedsApproval}\}$$

のように判定する。

外部文書由来の命令が「秘密ファイルを読み」「メール送信せよ」と主張しても、それは tool authorization を増加させない。これにより認識の失敗とシステム侵害を分離できる。

11 documentation.wl への適用

11.1 現行コードで確認できる数式経路

添付の `documentation.wl` では、インライン数式変換関数 `iDocConvertMathCellViaLLM` がセルを 144 dpi で Rasterize し、画像とプロンプトを VLM へ渡している。また `iDocDisplayMathToExport` は、まず `iDocBoxToTeX` によるプログラムの変換を試し、Box 式が残る場合に同様の画像 VLM フォールバックを行う。

代表的な処理は概念上、次に相当する。

```
img = Rasterize[cell, ImageResolution -> 144];  
result = iDocCallLLM[{img, prompt}, model];
```

この設計は、Box からの機械的変換だけでは復元しにくい式を視覚的に転写できる点で合理的である。DSCG 適用時もこの能力は捨てない。

11.2 推奨変更: 画像 VLM フォールバックの隔離

`iDocDisplayMathToExport` は次の順序にする。

1. `iDocBoxToTeX` を試す。
2. 成功した場合は数学 LaTeX validator へ通す。
3. 失敗した場合だけセルまたは数式領域を Rasterize する。
4. 背景 flatten、grayscale、2 値版を生成する。
5. FTS へ送り、math-only LaTeX を取得する。
6. strict parser で AST 化し、禁止 command がないことを確認する。
7. trusted renderer で再レンダリングし、必要に応じて round-trip 検証する。
8. 最終 export へ LaTeX のみを渡す。元画像は reasoning LLM へ渡さない。

11.3 インライン数式関数の改善

現行 `iDocConvertMathCellViaLLM` は「非数式テキストを正確に保持しつつ、数式を LaTeX 化した段落全文」を返す。この方式は実用上便利だが、安全境界としては出力文法が広い。

推奨は二段階である。

互換モード 現行の全文返却を維持するが、背景正規化、出力差分検査、「元 `plainText` 以外の自然言語文字列を追加していないか」の host-side validation を行う。

Strict モード VLM は数式 slot ごとの LaTeX だけを JSON で返す。本文は `NBCellGetText` または Notebook 構造を authoritative source としてホストが再構築する。

Strict モードでは、VLM 出力を自然言語として reasoning させないため、visual prompt injection が自由文へ展開される余地が小さくなる。

11.4 提案 API

実装上は、例えば次の内部関数へ分離するとよい。

```
iDocCanonicalizeDocumentImage[img_, opts___]
iDocCanonicalizeMathImage[img_, opts___]
iDocMathTranscribeSandbox[img_, opts___]
iDocValidateMathTeX[tex_String]
iDocParseMathAST[tex_String]
iDocVerifyMathRoundTrip[img_, tex_String]
iDocBackgroundAnomalyScore[img_]
iDocCanonicalizationConsensus[images_List]
```

特に `iDocMathTranscribeSandbox` は NBAccess 経由で LLM を呼ぶとしても、通常の文書生成 prompt、秘密セル、tool execution context を渡さない専用 profile にすべきである。

12 推奨アルゴリズム

12.1 ページ処理

```
function CanonicalizePage(pdf_page):
    I = TrustedRasterize(pdf_page, dpi=300)
    I = NormalizeColorSpace(I)

    bg = EstimateBackground(I)
    mask_bg = DetectFlatBackground(I, bg)
    anomaly = BackgroundAnomalyScore(I, mask_bg)
    I[mask_bg] = PURE_WHITE

    regions = SegmentDocument(I)
    outputs = []

    for r in regions:
        if r.kind == TEXT:
            outputs += OCR_and_Reconstruct(r)
        elif r.kind == DISPLAY_MATH:
            outputs += FormulaTranscriptionSandbox(r)
        elif r.kind == INLINE_MATH:
            outputs += FormulaSlotTranscription(r)
        elif r.kind == TABLE:
            outputs += ParseTable(r)
        elif r.kind in {DIAGRAM, PLOT}:
            outputs += QuantizeAndNormalize(r)
        elif r.kind == PHOTO:
            outputs += NormalizePhotoConservatively(r)

    return CanonicalDocument(outputs,
        provenance="external_document",
        anomaly_score=anomaly)
```

12.2 数式処理

```
function FormulaTranscriptionSandbox(math_crop):
    views = [
```

```

    FlattenBackground(ToGray(math_crop)),
    QuantizeGray(math_crop, bits=2),
    AdaptiveBinarize(math_crop)
]

latex = [MathOnlyVLM(v) for v in views]
ast = [StrictMathParse(x) for x in latex]

if any(parse_failed(ast)):
    return NEEDS_REVIEW

if not ASTConsensus(ast):
    return NEEDS_REVIEW

tex = CanonicalTeX(ast[0])
reconstructed = TrustedMathRender(tex)

if not RoundTripAccept(math_crop, reconstructed, ast[0]):
    return LOW_CONFIDENCE(tex)

return VERIFIED_MATH(tex)

```

13 評価方法

13.1 評価軸

DSCG を研究・実装として評価するなら、単純な attack success rate だけでなく、以下を同時に測る必要がある。

1. **Security:** hidden text、visual perturbation、OCR adversarial、adaptive attack に対する攻撃成功率。
2. **Utility:** OCR CER/WER、数式 exact/AST accuracy、表構造 F1、図理解性能、QA 精度。
3. **Semantic preservation:** 正常文書で要約・質問応答・査読結果がどれだけ変わらないか。
4. **False positive:** 正常なスキャン、背景色付き論文、watermark、低品質 PDF を不必要に隔離しないか。
5. **Cost:** ページ当たり処理時間、VLM 呼出し回数、GPU/CPU 負荷。
6. **Adaptive robustness:** 防御変換を既知とした攻撃で性能がどこまで低下するか。

13.2 ベンチマーク構成案

正常 PDF と、その PDF から生成した攻撃版を paired にするのがよい。背景摂動については同じ可視内容を保ったまま、背景だけに低周波・高周波・RGB 相関・bounded perturbation を加える。透明テキスト、極小文字、ページ外文字も別カテゴリにする。数式については攻撃成否だけでなく、 $\sum$ / $\prod$, $</>$, $0/O$, $1/l$, 添字範囲など、科学的意味を壊す微小転写誤りを個別に測る。

CrackedPDFs は hidden prompt injection の文書 aware 評価を目的とした最近のベンチマーク例であり [11]、DSCG 評価でも「攻撃版と benign confounder を対にする」考え方が参考になる。

13.3 Ablation

少なくとも次を比較する。

1. PDF text extraction のみ
2. rasterize + OCR のみ
3. rasterize + background flatten
4. + bit-depth/palette squeezing
5. + semantic region routing
6. + FTS
7. + multi-view consensus
8. + provenance/capability gate

これにより、どの防御層が security と utility へ寄与しているかを分離できる。

14 限界と Adaptive Attack

14.1 背景を消せば攻撃は foreground へ移る

背景を完全白化すれば、攻撃者は文字 edge、図の線、グラフ、写真領域へ摂動を移すと考えるべきである。従って背景 flatten は非常に有効な第一層だが、単独の完全防御ではない。

14.2 二値画像に対しても攻撃は理論上可能

二値化によって連続画素空間を大幅に縮めても、glyph の一部画素や形状を変更して OCR/VLM を誤らせる攻撃は可能である。ここでの価値は「不可能化」ではなく、**攻撃者の自由度を文書可読性と強く競合する領域まで押し込む**ことである。攻撃が人間にも目立つようになれば、少なくとも不可視性という優位を失う。

14.3 transform-aware attack

JPEG、resize、二値化を固定すると、攻撃者はそれらを学習ループに含めて摂動を最適化できる。対策として、許容範囲内のランダム化、複数変換 consensus、異なる OCR/VLM の ensemble、再レンダリングによる構文ボトルネックを組み合わせる。ただしランダム化だけを安全根拠にしない。

14.4 写真の semantic corruption

写真は高次元性を残すため、最も強い残存リスクである。しかも purification が科学的信号を失わせる可能性がある。従って写真については、「画像を安全に変換する」より、**画像から得た記述は untrusted evidence である**という provenance 管理と、重要判断時の原画像確認を組み合わせる必要がある。

14.5 認識器の共通モード失敗

複数変換や複数モデルがすべて同じ誤認識をする可能性はある。multi-view consensus は信頼度推定を改善するが、正しさの証明ではない。特に数式では、重要な式を原論文画像と対照できる UI を残す価値が高い。

15 実装優先順位

15.1 Phase 1: 低コスト・高効果

1. PDF text layer を reasoning LLM へ直接投入しないモードを追加。
2. trusted rasterize を標準経路にする。
3. ページ背景を検出して純白化。
4. OCR text と画像由来結果に `untrusted_document` provenance を付与。
5. 文書由来の指示で tool permission が増えない capability rule を実装。

15.2 Phase 2: 数式経路の強化

1. `iDocDisplayMathToExport` の画像フォールバックへ `background flatten` を追加。
2. `math-only` prompt profile を分離。
3. LaTeX command whitelist / parser を追加。
4. grayscale・2-bit・binary の multi-view 転写と AST consensus を実装。
5. trusted renderer による round-trip 検証を追加。

15.3 Phase 3: 文書全体の semantic routing

表・図・写真の領域分割、palette quantization、表構造化、写真用 consistency check を追加する。ここまで来ると DSCG は単なるサニタイザではなく、**安全な document ingestion layer** として独立コンポーネント化できる。

16 方式の利点とトレードオフ

観点	利点	トレードオフ / 注意点
透明 PDF 文字	rasterize で不可視 text layer を切断しやすい	可視文字として描画された攻撃は残る
背景摂動	flat background を 1 色へ潰せるため非常に相性がよい	scan/紙質/透かしとの識別が必要
VLM 敵対的摂動	bit-depth 削減・量子化で入力自由度を減らせる	adaptive attack を完全には防げない
数式	画像認識精度を残しつつ LaTeX/AST へ狭く圧縮できる	parser と round-trip 検証の実装コスト
表	構造データへ落とし込みやすい	複雑な結合セル・数式表で難しい
写真	標準再エンコードで一部の摂動を弱められる	科学的情報を壊さない範囲が狭い
Agent 安全性	capability gate で認識失敗と侵害を分離できる	ツール権限設計をアプリ側で実装する必要
監査性	canonical form と原文を分離保存し差分監査可能	保存容量・provenance 管理が増える

17 結論

論文・レポートを LLM へ投入する場合、画像化は透明テキスト型 prompt injection を減らす有力な手段だが、画像をそのまま VLM へ渡すと visual adversarial example という別の攻撃面が現れる。一方、学術文書は一般画像と異なり、背景や文字など多くの領域が低エントロピーである。この特性を利用し、**背景を完全平坦化し、領域ごとに必要な情報量だけ残す**ことは、攻撃者が利用できる signal subspace を大幅に削減する合理的な防御になる。

特に数式については、単純な OCR・再レンダリングだけでは品質を保てない。そのため、**元数式画像を一度だけ隔離された math-only VLM へ提示し、出力を安全な LaTeX/数学 AST へ圧縮した後、trusted renderer で再構成する**方式が実用上の妥協点となる。この方式なら、documentation.wl で現在利用している「画像を直接見せることで数式変換精度を上げる」利点を残しながら、元画素が最終 reasoning/agent LLM へ流入する経路を切断できる。

最も重要なのは、DSCG を「万能な攻撃検出器」と位置づけないことである。背景 flatten、feature squeezing、multi-view consensus、FTS は入力攻撃面を狭める。最終的なシステム安全性は、文書由来情報を常に untrusted として扱い、**capability gate によって権限を別管理**することで担保する。この二層構造により、未知の visual prompt injection に対しても、攻撃成功が直ちにシステム侵害へつながらない設計が可能になる。

A 数学 LaTeX サブセットの設計例

実装時には「禁止リスト」より「許可リスト」を基本とする。以下は概念例であり、実際には parser ベースで AST を構築することを推奨する。

分類	原則許可	原則拒否
構造	<code>\frac</code> , <code>\sqrt</code> , <code>\int</code> , <code>\{</code> , <code>\}</code>	マクロ定義、カテゴリコード変更
演算子	<code>\sum</code> , <code>\prod</code> , <code>\int</code> , <code>\lim</code>	外部プログラム実行に関わる機能
配列	限定された <code>matrix/aligned/cases</code>	任意 environment
文字	数字、Latin/Greek、数学記号	制御文字、不可視特殊文字
ファイル I/O	なし	<code>\input</code> , <code>\include</code> , <code>\openout</code> , <code>\write</code>
外部参照	なし	URL、画像、shell escape、special

B 実装チェックリスト

1. rasterizer は LLM/agent プロセスから分離されているか。
2. PDF text layer を直接 reasoning LLM へ入れる経路が残っていないか。
3. 背景 flatness の統計を保存できるか。
4. 数式画像 VLM は tool access と秘密 context を持たないか。
5. FTS 出力を自由自然言語として後段へ渡していないか。
6. LaTeX は strict AST parser を通るか。
7. 元画像と canonical image の両方を監査用に保存できるか。
8. canonicalization version と parameters を provenance へ記録しているか。
9. 文書由来命令で capability が昇格しないか。
10. 正常文書での数式・表・図の utility regression を継続測定しているか。
11. adaptive attack を含む red-team corpus を CI で再評価できるか。

C 参考文献

References

- [1] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, M. Fritz, “Not what you’ve signed up for: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection,” arXiv:2302.12173, 2023. <https://arxiv.org/abs/2302.12173>
- [2] X. Qi, K. Huang, A. Panda, P. Henderson, M. Wang, P. Mittal, “Visual Adversarial Examples Jailbreak Aligned Large Language Models,” arXiv:2306.13213, 2023; AAAI 2024. <https://arxiv.org/abs/2306.13213>
- [3] L. Bailey et al., “Image Hijacks: Adversarial Images can Control Generative Models at Runtime,” arXiv:2309.00236, 2023. <https://arxiv.org/abs/2309.00236>
- [4] W. Xu, D. Evans, Y. Qi, “Feature Squeezing: Detecting Adversarial Examples in Deep Neural Networks,” arXiv:1704.01155, 2017. <https://arxiv.org/abs/1704.01155>
- [5] L. Chen et al., “Attacking Optical Character Recognition (OCR) Systems with Adversarial Watermarks,” arXiv:2002.03095, 2020. <https://arxiv.org/abs/2002.03095>

- [6] C. Pathade, “Invisible Injections: Exploiting Vision-Language Models Through Steganographic Prompt Embedding,” arXiv:2507.22304, 2025. <https://arxiv.org/abs/2507.22304>
- [7] M. Ding, S. Xia, C. Kong, X. Jiang, “Adversarial Prompt Injection Attack on Multi-modal Large Language Models,” arXiv:2603.29418, 2026. <https://arxiv.org/abs/2603.29418>
- [8] J. Fu, Y. Wu, Y. Chen, K. Peng, X. Zhang, V. Cevher, S. Pashami, A. Holst, “DiffCAP: Diffusion-based Cumulative Adversarial Purification for Vision Language Models,” arXiv:2506.03933, 2025. <https://arxiv.org/abs/2506.03933>
- [9] J. Keuper et al., “Prompt Injection Attacks on LLM Generated Reviews of Scientific Manuscripts,” arXiv:2509.10248, 2025. <https://arxiv.org/abs/2509.10248>
- [10] “TrapDoc: Deceiving LLM Users by Injecting Imperceptible Prompts in PDF Documents,” arXiv:2506.00089, 2025. <https://arxiv.org/abs/2506.00089>
- [11] P. Thienpreecha, “CrackedPDFs: A Controlled Benchmark for Hidden Prompt Injection in PDFs,” arXiv:2607.19396, 2026. <https://arxiv.org/abs/2607.19396>

本レポートは 2026 年 8 月 11 日時点の公開研究と、添付された `documentation.wl` の実装内容、および本会話で検討した設計案を統合して作成した。